

Active-set methods for smallest enclosing balls: invariance conditions and a controlled comparison of primal and dual

Thomas Schmelzer

September 6, 2026

Abstract

Two active-set methods solve the smallest enclosing ball problem from opposite sides: the dual ascent over the unit simplex of Dearing and Zeck [3], which holds no enclosing ball until it terminates, and the primal deflation of Fischer, Gärtner and Kutz [5], which is feasible at every step. Neither method is ours. They keep the same support set, solve the same subproblem, and can repair the same QR factorisation of the same edge matrix in $O(dr)$. We give the conditions under which either survives floating point, measure what each is worth, and then hold that linear algebra fixed to see what separates the two methods.

The conditions are sharper than tolerance hygiene. Each method carries exactly one tolerance with units of length and one with none, and the two must be sized differently; the affine-dependence test must run on the edge matrix rather than on the system bordered by a row of ones, which would call any cloud of small extent degenerate; and since a *squared* double reaches only 10^{154} , the centred cloud must be rescaled by an exact power of two — any other factor rounds, and the exactness goes with it.

With those in place both methods return a ball whose enclosure error is a few ulp on Gaussian clouds to $d = 1.6 \cdot 10^4$, eight times the largest dimension previously reported for the pivoting method, agreeing on the support set on every row; the support grows as $\sqrt{d}$ rather than as d , and time with it as $d^{3/2}$. What remains between the two is a factor of 1.2 to 2.1 in steps — the price of the primal method never taking a full step — which the wall clock damps to between 1.1 and 1.6, and a difference in what each can hand back: dual weights that certify optimality in one pass, against a ball that is already valid if the iteration is cut short.

1 Introduction

Sylvester stated the problem in 1857 in a single sentence [12]: given a finite set of points in the plane, find the smallest circle containing all of them. It has been a standard example ever since — short enough to state in a line, geometric enough to draw, and just hard enough that the obvious algorithms are wrong. It is the 1-centre facility location problem, where one facility is placed so that the farthest customer travels as little as possible. It is the primitive under bounding-sphere hierarchies in graphics and collision detection. As the minimum enclosing ball of a set of feature vectors it is the core-set problem behind kernel machines. And it is a fixture of solver documentation: Hexaly ships it as a modelling template [8], CGAL as `Min_sphere_d` — Welzl’s recursion for small inputs, Gärtner’s method for large ones.

The algorithmic history is largely a computational-geometry one. Welzl’s randomised incremental method [13] solves it in expected $O(n)$ time for fixed d by recursing over subsets with a few points forced onto the boundary, with a constant that grows superexponentially in d ; much of what followed is a response to that. Gärtner [6] made the recursion’s predicates robust in floating point, Gärtner and Schönherr [7] recast the problem as a quadratic program and solved it exactly, reserving judgment past a few hundred dimensions, and Fischer, Gärtner and Kutz [5] left the recursion behind for a pivoting method that deflates an enclosing ball, carrying a dynamic QR of the support’s edge matrix and reporting d in the thousands — a finite algorithm of that second kind going back to Elzinga and Hearn [4].

A second line runs beside it in the optimization literature and reaches the same object from the dual. Dearing and Zeck [3] gave a dual algorithm for the ball of points, maintaining a support set and raising the radius to R from below; Cavaleiro and Alizadeh [1] brought its iteration from $O(d^3)$ to $O(d^2)$ by updating a factorisation of that support rather than rebuilding it, and then extended the method to balls [2], treating the affinely dependent case explicitly and reporting $d = 5 \cdot 10^3$ at $n = 10^5$. The two lines have largely stayed apart.

What returns the problem to an optimizer’s desk is that its dual is a concave quadratic over the unit simplex: the active set is the set of points on the boundary sphere, and the certificate is geometric rather than algebraic — the points carrying weight are exactly the points at distance R , which a caller can check in one pass. Active-set machinery is less comfortable on feasible sets with more structure [10, 11]; here there is none to speak of.

Two things follow. The first is a set of conditions under which either method survives floating point, in Section 6: which of each method’s two tolerances carries units of length and which carries none, why the

affine-dependence test must run on the edge matrix rather than on the system bordered by a row of ones, and why the rescaling that keeps a squared double inside the exponent range has to be by an exact power of two. They are stated for both methods because both need them, and they are what lets the measurements below reach the dimensions they do.

The second is a comparison with the linear algebra held fixed. The dual ascent of Section 3 and the primal deflation of Section 4 — the pivoting method of Fischer, Gärtner and Kutz [5] — keep the same support set, solve the same subproblem and repair the same factorisation (Section 5). Handing both the same data structure removes the usual confound, and what is left is a property of the two searches rather than of two implementations: on every input measured in Section 8 they agree on the ball and on its support, and separate by about a factor of two in steps and by less in time. Where they part is in what they hold while they run, and so in what they can promise if interrupted.

2 The problem and its two sides

For $S = \{p_1, \dots, p_n\} \subset \mathbb{R}^d$ the smallest enclosing ball is the solution of

$$\min_{x,r} r \quad \text{s.t.} \quad \|p_i - x\| \leq r, \quad i = 1, \dots, n, \quad (1)$$

a second-order cone program in $(r, x) \in \mathbb{R}^{1+d}$ with n blocks $(r, p_i - x) \in \mathcal{Q}^{d+1}$. Existence follows from coercivity of $f(x) = \max_i \|p_i - x\|$ and uniqueness from strict convexity of f^2 along any segment; we write (R, x^*) for the optimum.

Let $\Delta_n = \{u \in \mathbb{R}^n : u \geq 0, \mathbf{1}^\top u = 1\}$, and for $u \in \Delta_n$ put $x(u) = \sum_i u_i p_i$ and

$$\varphi(u) = \sum_i u_i \|p_i\|^2 - \|x(u)\|^2, \quad (2)$$

a concave quadratic. Everything below rests on one identity, valid for all $y \in \mathbb{R}^d$ and $u \in \Delta_n$:

$$\sum_i u_i \|p_i - y\|^2 = \sum_i u_i \|p_i\|^2 - 2x(u)^\top y + \|y\|^2 = \varphi(u) + \|x(u) - y\|^2. \quad (3)$$

Both steps are mechanical: expand and collect, using $\sum_i u_i p_i = x(u)$ on the cross term and $\mathbf{1}^\top u = 1$ on $\|y\|^2$, then add and subtract $\|x(u)\|^2$. It is the parallel-axis identity — measuring from anywhere other than the weighted mean costs precisely $\|x(u) - y\|^2$, whatever the weights, and $\varphi(u)$ is what is left when y is the mean.

Proposition 1. $R^2 = \max_{u \in \Delta_n} \varphi(u)$, and $x^* = x(w)$ for any maximiser w .

Both directions are short: $\varphi \leq R^2$ is (3) applied to any enclosing ball, and equality follows by separating x^* from $\text{conv } T$, $T = \{p_i : \|p_i - x^*\| = R\}$, if the two were disjoint. See also [7].

Since $\partial\varphi/\partial u_i = \|p_i - x(u)\|^2 - \|x(u)\|^2$, the first-order conditions for $\max_{\Delta_n} \varphi$ read, after cancelling the common term,

$$\|p_i - x\| = R \quad (w_i > 0), \quad \|p_i - x\| \leq R \quad (w_i = 0), \quad (4)$$

the geometric certificate: the points carrying weight lie on the boundary sphere and the rest are enclosed. By Carathéodory the support may be taken of size at most $d + 1$, so (1) is determined by at most $d + 1$ of the n points and the problem is combinatorial in the choice of that subset.

Read forwards, (4) is a search for weights: keep a set of points on the sphere, adjust the weights, stop when no point lies outside. Read backwards it is a search for balls: keep a ball containing S with a set T on its boundary, and shrink it until its centre falls inside $\text{conv } T$. The second rests on a fact due to Seidel, recorded in [7] and Lemma 1 of [5]: if T lies on the boundary of a ball B centred at c , then B is the smallest ball enclosing T if and only if $c \in \text{conv } T$. With $S \subseteq B$ and $T \subseteq S$ the inclusions $\text{seb}(T) \subseteq \text{seb}(S) \subseteq B$ close, so a ball containing the cloud, carrying T on its boundary and centred inside $\text{conv } T$ is *the* answer — and $\text{conv } T$ is the only thing left to check. Sections 3 and 4 are those two readings made into methods. They keep the same kind of state — a subset of S that will end up on the boundary, and a point in its affine hull — and differ in which feasibility they refuse to give up. The first stays dual feasible, so by Proposition 1 its radius rises to R from below and its ball encloses nothing until the last iteration; the second stays primal feasible, so its radius falls to R from above and every iterate is an answer, merely not yet the best one.

3 The dual active-set method

This method is not new and is not claimed here. It is the dual algorithm of Dearing and Zeck [3] as sharpened by Cavaleiro and Alizadeh [1, 2], restated because Section 4 is to be compared against it and the correspondence

must be exact, and because Section 6 concerns tolerances that mean nothing until the steps they guard are on the page. Two details below depart from that line of work, and are given as differences rather than as claims: the step rule here is a full step to the circumcentre of the current face followed by a ratio test (7), where [2] performs an exact curve search that keeps dual feasibility throughout; and an affinely dependent support is resolved by a descent direction in the null space rather than by dropping a point before the search begins.

Maintain a working set F and weights $u > 0$ on it supported there and summing to one:

$$u \in \Delta_n, \quad \text{supp}(u) \subseteq F, \quad \varphi(u) \text{ non-decreasing}, \quad (5)$$

the dual counterpart of the invariant (9) carried by the method of Section 4. This is a primal active-set method run on the dual (2): the free variables are the weights of the support, every subproblem solves one face exactly, and feasibility for the original problem arrives only at the end — by Proposition 1, $\sqrt{\varphi(u)}$ is a lower bound on R at every iterate, so the ball on hand is too small until the last one.

Subproblem. Maximising φ over weights supported on F with no sign restriction is, by (4) without the inequalities, the requirement that x be equidistant from $\{q_j\}_{j \in F}$ and lie in their affine hull. With D the $(m-1) \times d$ matrix of edges $e_j = q_j - q_0$ and $x = q_0 + D^T y$,

$$(DD^T)y = \frac{1}{2}(\|e_1\|^2, \dots, \|e_{m-1}\|^2)^T, \quad w = \left(1 - \sum_j y_j, y\right), \quad (6)$$

a dense system of order $m-1 \leq d$, non-singular exactly when F is affinely independent. This is the circumcentre of the face, expressed in barycentric coordinates.

Dropping. If $\min_j w_j < 0$ the circumcentre lies outside $\text{conv}\{q_j\}$ and the subproblem solution is dual-infeasible. Move along $s = w - u$ by the ratio test

$$\alpha = \min\{-u_i/s_i : s_i < 0\}, \quad (7)$$

which drives at least one weight to zero; remove it from F .

Degeneracy. If F is affinely dependent, DD^T is singular. The directions s with $\mathbf{1}^T s = 0$ and $\sum_i s_i q_i = 0$ — equivalently $D^T s_{1:} = 0$ after eliminating s_0 — leave $x(u)$ fixed, so $\|x(u)\|^2$ is frozen and $\varphi(u + ts) = \varphi(u) + t \sum_i s_i \|p_i\|^2$ is linear: the subproblem is unbounded. Project the gradient onto that null space and apply (7), shrinking F by one.

Adding. Otherwise accept $u \leftarrow w$, set $x = x(u)$ and $R = \max_{i \in F} \|p_i - x\|$, and let $k = \arg \max_i \|p_i - x\|$. If $\|p_k - x\| \leq R$ then (4) holds and (R, x, u) is optimal; otherwise append k to F with weight zero. By (8) below this is also the steepest-ascent choice among the fixed variables.

Each iteration either strictly increases φ or strictly shrinks F , and there are finitely many working sets, so the method is finite; a degenerate zero-length step needs an anti-cycling rule in exact arithmetic, and in practice an iteration cap serves as the safety net. Note that

$$\partial\varphi/\partial u_k - \partial\varphi/\partial u_i = \|p_k - x\|^2 - \|p_i - x\|^2, \quad (8)$$

so the farthest violator is the fixed variable with the largest reduced gradient.

4 The primal pivoting method of Fischer, Gärtner and Kutz

The method of Fischer, Gärtner and Kutz [5] carries a ball rather than a set of weights, and never surrenders it. Its state is a pair (c, T) , a centre and a support set, subject to the invariant of their Fig. 2:

$$S \subseteq B(c, T), \quad T \subseteq \partial B(c, T), \quad T \text{ affinely independent}, \quad (9)$$

where $B(c, T)$ is the ball about c through the farthest point of T . By Seidel's lemma the pair is optimal as soon as $c \in \text{conv } T$, and c is then the circumcentre $\text{cc}(T)$ without further argument. Each iteration is a dropping phase, entered only once c has landed in $\text{aff}(T)$, followed by a walking phase.

Dropping. $c \in \text{aff}(T) \setminus \text{conv } T$ means some affine coefficient of c with respect to T is negative; drop such a point. The coefficients are the same object as the barycentric weights of (6) and come from the same factorisation (Section 5), but here they are read rather than moved: there is no ratio test, because c does not travel during a drop. Their Lemmata 4 and 5 together say the dropped point cannot stop the walk that immediately follows, which is what keeps the two phases from undoing each other.

Walking. Move c along $e = cc(T) - c$. Their Lemma 3 says that segment is orthogonal to $\text{aff}(T)$, that T stays on the boundary the whole way, and that the radius strictly decreases — the deflation. Two consequences make the step cheap. Orthogonality makes $cc(T)$ the projection of c onto $\text{aff}(T)$, so no circumsphere is ever solved for and one projection does the work of (6). And the stopping fraction has a closed form: with $c(a) = c + ae$, a point p joins the boundary when $\|p - c(a)\|$ equals the common support distance; the a^2 terms cancel, and $\langle q - c, e \rangle = \|e\|^2$ for every $q \in T$ leaves

$$a_p = \frac{R_c^2 - \|p - c\|^2}{2(\|e\|^2 - \langle p - c, e \rangle)}, \quad R_c = \max_{q \in T} \|q - c\|, \quad a = \min\{1, \min_p a_p\}. \quad (10)$$

Criterion (1) of Lemma 4 — that p lies behind $\text{aff}(T)$ and so cannot stop the walk — is exactly the statement that this denominator is non-positive, so one sign test separates the candidates and the numerator is non-negative on the rest by (9). If $a = 1$ the centre arrives at $cc(T)$, lands in $\text{aff}(T)$, and the next iteration drops; otherwise the minimising point enters T and the ball has shrunk.

Pivot rules. Ties are resolved two ways. Bland’s rule, adapted — fix an order on S , take the smallest-rank candidate at both the drop and the admission — is what their Theorem 1 proves terminating, degeneracies included, and is also recorded there as slow. The heuristic, which their own code runs and which the table below reports, drops the *most* negative coefficient and admits the tied point farthest from $\text{aff}(T)$, leaving the support best conditioned.

The radius falls strictly on every walk and no configuration recurs, so the method is finite; under Bland’s rule that is Theorem 1, and under the heuristic an iteration cap serves as the safety net, exactly as in Section 3.

Degeneracy, and the price of the invariant. The invariant (9) requires T affinely independent, and there is no fallback if it is not: with $\text{aff}(T)$ of the wrong dimension the projection that defines the walk is meaningless. So the entering point must be certified off $\text{aff}(T)$ before it is admitted — the paper’s stability threshold, and the one tolerance in the method carrying units of length (Section 6). Section 3 instead *permits* a dependent support and answers it with a descent direction in the null space, paying a rank test every iteration and a factorisation that must survive a state an economic QR cannot represent. Neither choice dominates — one avoids the case, the other resolves it.

How the two differ. The support sets agree and the drop criteria agree — a negative barycentric coordinate, in both — and the mechanics agree nowhere else. Section 3 takes the whole step to $cc(F)$ and only then consults the data, admitting the farthest violator; this method never takes a full step while any point is still outside, truncating the walk at whichever point first touches the shrinking sphere, which is also the point that enters. A drop here is immediate and followed by a resumed walk; there it is a ratio test in weight space, moving every weight at once. That difference is what Section 8 measures once the linear algebra is held fixed.

5 The support’s factorisation

Per iteration, both methods want the same two things from their support: whether it is affinely independent, and where the circumcentre of its face lies. Both read those off the same object. Let D be the $r \times d$ matrix of edges $q_j - q_0$ of the current support and let $D^\top = QR$ be its economic factorisation. Three identities then take every per-iteration quantity off d and onto the $r \times r$ block:

$$DD^\top = R^\top R, \quad \ker D^\top = \ker R, \quad \|q_j - q_0\|^2 = \|Re_j\|^2. \quad (11)$$

The first makes (6) two triangular solves against a Cholesky factor that is already to hand, with no $O(r^2d)$ product; the second makes the affine-dependence test of Section 3 a rank test on R — note it is the *right* null space, not the one the edge matrix itself would hand you; the third supplies the right-hand side. The pivoting method reads the same R for its projection and its coefficients.

Rebuilding that factorisation costs $O(dr^2)$ and repairing it costs $O(dr)$, and the repair is section 4 of [5], and the same move on the dual side is [1]. Three updates cover every move either method makes: a point joining the support appends a column, a point other than q_0 leaving deletes one, and q_0 leaving has no column of its own — every column being measured *from* it — so q_1 is promoted by a deletion followed by a rank-one update. In a compiled language these are Givens sweeps; in an interpreted one they have to be the library’s, or a single vectorised refactorisation will beat them and the measurement will be of the interpreter rather than of the data structure.

Two things keep this from being a pure win. The dependent supports Section 3 permits are exactly what an economic QR cannot hold — an update refuses a column already in the span — so those faces are met by

refactorising, affordable only because the surrounding tolerances keep them rare. And the repair is not cheaper at every size: on the clouds of Section 8 it is worth a factor of 3.5 at $d = 8000$, is at parity near $d = 100$, and loses below that. An implementation therefore dispatches on d rather than committing to one route. That threshold is why Section 8 can report $d = 1.6 \cdot 10^4$ at all, and why a cloud of low dimension is indifferent to it.

6 Invariance and floating point

Two properties are load-bearing for either method and easily lost by both. Both are consequences of refusing any tolerance tied to coordinates of magnitude one.

Origin invariance. Recentre on $\bar{p}$ before iterating, and form $\|p - x\|^2$ as $\sum_k (p_k - x_k)^2$ rather than $\|p\|^2 - 2p^\top x + \|x\|^2$. The expanded form cancels quantities of size $\|x\|^2$ to produce an answer of size R^2 , losing every digit by which $\|x\|$ exceeds R ; for a cloud of unit extent at distance 10^6 the rounding floor of a squared distance is then of the order of the radius itself.

Scale invariance. Test affine dependence on the rank of D , not on $[\mathbf{1}; Q^\top]$: the row of ones has unit entries, dominates the singular values, and would declare any cloud of extent below ε degenerate. Size the feasibility slack of the stopping test as a relative tolerance plus an absolute floor proportional to $\varepsilon \max_i \|p_i\|^2$, and normalise the null-space direction before comparing anything derived from it — the gradient carries units of length², the weight tolerance does not.

Which tolerance carries units. The two methods have one apiece, and they are not the same one. Section 3 must decide when a weight has reached zero, which is dimensionless, and when a null-space direction is binding, which is not — hence the normalisation above. Section 4 must decide when a candidate is far enough off $\text{aff}(T)$ to be admitted, which carries units of length and is sized off the cloud's extent; its other tolerance, on the affine coefficients, needs no such care, being dimensionless and bounded by $1/2$ in the optimal configuration (Lemma 2 of [5]). The pattern is the same in both: the dimensionless test gets a constant, the dimensioned one gets the extent.

Exponent range. Tolerances are not the whole of it. A double reaches to 10^{308} , so a *squared* double reaches only to 10^{154} , and both methods square everything: the Gram matrix, the distances, the noise floor. A cloud of extent 10^{-160} — ordinary coordinates — therefore has a Gram matrix in the subnormals, and the solve (6) overflows; at 10^{+160} the squares saturate and a method reports $R = 0$ without raising. Rescaling the centred cloud to unit extent removes both, and the factor must be a power of two: any other rounds, and the exactness Table 2 reports is lost with it. The measurements below rescale in the dual method and not in the pivoting one, which is a property of the two implementations rather than of the two methods — the walk of (10) would survive the same treatment unchanged.

Table 1 is the same method with each of those guards removed in turn. Only the fully guarded row is exactly zero everywhere exactness is available. In the two offset columns it sits instead at $\varepsilon \|\bar{p}\|/R$, the floor imposed by storing the answer in coordinates of that magnitude at all — a property of the representation, not of the search, and the measurement itself has to be recentred and rescaled before squaring or it reports that floor as a failure of the method.

guards in place	extent 1, offset			at the origin, extent		
	0	10^6	10^9	10^{-20}	10^{-160}	10^{160}
all	0	$5.4 \cdot 10^{-11}$	$6.5 \cdot 10^{-8}$	0	0	0
expanded distances	$2.2 \cdot 10^{-16}$	$5.4 \cdot 10^{-11}$	$6.5 \cdot 10^{-8}$	$2.2 \cdot 10^{-16}$	$2.2 \cdot 10^{-16}$	$2.2 \cdot 10^{-16}$
no recentring	$2.2 \cdot 10^{-16}$	$5.3 \cdot 10^{-2}$	$R = 0$	$2.2 \cdot 10^{-16}$	$2.2 \cdot 10^{-16}$	$2.2 \cdot 10^{-16}$
no rescaling	0	$5.4 \cdot 10^{-11}$	$6.5 \cdot 10^{-8}$	0	non-finite	$R = 0$
and unit tolerances	0	$5.4 \cdot 10^{-11}$	$6.5 \cdot 10^{-8}$	$R = 0$	$R = 0$	stalls
rescale by the extent	$2.2 \cdot 10^{-16}$	$5.4 \cdot 10^{-11}$	$6.5 \cdot 10^{-8}$	$2.2 \cdot 10^{-16}$	$2.2 \cdot 10^{-16}$	$2.2 \cdot 10^{-16}$
none	$2.2 \cdot 10^{-16}$	$1.5 \cdot 10^{-3}$	stalls	$R = 0$	$R = 0$	stalls

Table 1: Relative enclosure error $\max_i \|p_i - x\|/R - 1$ for the method of Section 3 with individual guards removed: $n = 10^3$ standard normal points in $d = 10$, scaled and shifted per column, worst of 15 seeds. $R = 0$ means the method returned a radius of zero and raised nothing; *non-finite* that the weights left the reals; *stalls* that no support point blocked the step. Of 48 cells, 42 fall in the same category on every seed — the six that do not are all failure rows differing in *how* they fail.

The two ends behave differently, and the dangerous one is the top. Without rescaling, a cloud of extent 10^{-160} drives the weights out of the reals and the method raises; at 10^{160} it returns $R = 0$ and raises nothing, which is the failure a caller cannot detect. Tolerances tied to magnitude one bind far earlier — already at extent 10^{-20} , where the stopping test accepts the first iterate and reports $R = 0$. And rescaling by the extent

rather than by a power of two costs exactly what Section 6 claims it costs: one ulp, everywhere the power of two is exact.

One row qualifies the section rather than confirming it. Expanding $\|p - x\|^2$ instead of differencing costs nothing measurable *once the cloud is recentred*, because recentring already leaves $\|x\|$ of the order of R and there is then nothing to cancel. The two halves of origin invariance are not independent: recentring is the load-bearing one, and differencing is what makes its absence merely bad rather than catastrophic. Differencing is free, so both are kept, but only the first is doing work on these inputs.

7 Cost, and what each method certifies

Per iteration each method performs one $O(nd)$ sweep of the cloud — squared distances to the centre in Section 3, the stopping fractions (10) in Section 4 — and one solve or projection of order at most d . Nothing of size n is ever factorised, which is the structural difference from an interior-point method applied to (1), and both solve the final face exactly rather than leaving the caller to decide which slacks of size 10^{-9} were meant to be zero.

What they can hand back differs, and it is the practical reason to keep both. Section 4 is primal feasible at every step: interrupt it at any iteration and what is on hand is an enclosing ball, correct if not yet smallest, with a radius that only falls from here. That is the guarantee to want under a time budget, and Section 3 cannot offer it — by Proposition 1 its radius is a lower bound throughout, so its ball fails to contain the cloud until the final iteration. In exchange it finishes with the dual weights.

Those weights are the conic dual of (1) after one rescaling — $z_i = u_i(1, -(p_i - x)/R)$, which lies in $\mathcal{Q}^{d+1}$, is tight exactly on the support, and pairs with the slack $s_i = (R, p_i - x)$ to give $s_i^\top z_i = 0$. A solver built on this method therefore returns a standard-form primal–dual pair, and a caller can verify optimality from (4) in one pass over the data, without re-solving. That is what makes it usable behind a conic interface rather than only as a special-purpose routine; Section 4 answers the same program but would have to solve (6) once at the end to produce the weights.

8 A numerical comparison

Table 2 fixes $n = 10^3$ and takes d from 10^3 to $1.6 \cdot 10^4$, the regime [5] was written for and the first in which the linear algebra rather than the combinatorics decides the cost. Two quantities are reported, because either alone would mislead. The first is the relative enclosure error $\max_i \|p_i - x\|/R - 1$, a deterministic property of the returned pair, free of timing noise, where a positive value means the ball does not in fact contain the cloud. The second is wall-clock time, which must be read as a comparison of *implementations*: both methods spend most of each iteration in one vectorised sweep and one repair of the factorisation, so the step counts beside them are what carry over to another language.

d	steps			enclosure error		time (s)			
	support	act. set	pivoting	act. set	pivoting	act. set	pivoting	ratio	
1 000	92	92	197	$-4.4 \cdot 10^{-16}$	0	0.148	0.232	1.56	
2 000	124	128	208	$-1.1 \cdot 10^{-16}$	$-1.1 \cdot 10^{-16}$	0.302	0.355	1.18	
4 000	177	177	249	$-6.7 \cdot 10^{-16}$	0	1.296	1.450	1.12	
8 000	217	217	328	$-1.0 \cdot 10^{-15}$	0	2.760	3.272	1.19	
16 000	283	283	352	$-1.4 \cdot 10^{-15}$	0	7.869	8.457	1.07	

Table 2: Growth in d at $n = 10^3$: standard normal clouds, one per row, median of three runs. Apple M4 Pro, CPython 3.12.13, NumPy 2.5.1, SciPy 1.18.0. Both methods repair the QR of their support’s edge matrix rather than rebuilding it, and both identify the same support set, whose size is the first column. *Steps* are active-set iterations against pivot steps, which count comparable work here: one solve or projection of order r plus one $O(nd)$ sweep each. Welzl’s method and Clarabel are not run here: neither finishes in this regime.

Neither reference appears in the table. Welzl’s median basis count is already $2.8 \cdot 10^5$ at $d = 11$, and Clarabel would be handed a program of $n(d + 1) \approx 1.6 \cdot 10^7$ rows to factor. Both methods that remain repair the factorisation of their support rather than rebuilding it (Section 5), which is what lets either appear here at all.

An active-set iteration and a pivot step each cost one solve or projection of order r plus one $O(nd)$ sweep, so the two step counts are comparable here. They separate by about a factor of two, in the direction the mechanics predict. The active-set count equals the final support size on four of the five rows — on Gaussian data the method adds points and essentially never drops one — while the pivoting method takes 1.2 to 2.1

times as many steps, the price of never taking a full step: each walk is truncated by the first point of S to touch the shrinking sphere. The wall clocks do not separate by anything like that factor. Their ratio sits between 1.1 and 1.6 with no trend in d , and repeat runs move it by a few percent, so it is a small constant rather than a curve: with r in the hundreds both methods spend most of each iteration in the same two places, and the pivoting method's extra steps are its cheapest ones.

Two further things are worth naming. The support grows like $\sqrt{d} - 92$ points at $d = 10^3$ and 283 at $d = 1.6 \cdot 10^4$ — staying an order of magnitude below the Carathéodory bound $d + 1$, so the per-iteration solve is of that order rather than of d ; with $O(\sqrt{d})$ iterations of $O(nd)$ each the total should scale as $d^{3/2}$, and it does, a 53-fold increase in time over a 16-fold increase in d . And the enclosure errors stay at a few ulp for both methods across four doublings of d : the exactness is a property of terminating at a basis, not of small d .

The table runs past the $d = 2000$ of [5] with the same support on every row, and both columns were measured with the factorisation repaired rather than rebuilt — so the ratio between them measures what the two searches cost and not what two data structures cost. Neither is a tuned implementation, and their agreement to within a small constant factor is therefore a claim about the methods rather than about the code.

9 Notes

Welzl's recursion searches the same basis combinatorially, driven by a random permutation, with the analysis of LP-type problems behind it; both methods here search by pivoting instead, monotonically and deterministically, and claim no expected-time bound in exchange. If all points share a norm — after the standard lift of a kernel problem onto a sphere, say — then (2) reduces to $\varphi(u) = \rho^2 - \|x(u)\|^2$ and Section 3 becomes the minimum-norm point method of Wolfe [14], its working set his corral and (6) his affine minimiser; the linear term $\sum_i u_i \|p_i\|^2$ is what makes the subproblem a circumcentre rather than a projection.

Of the two lines of work Section 1 sketches, the method of Section 4 is the geometric one and that of Section 3 the quadratic-programming one, running from Gärtner and Schönherr [7] through Dearing and Zeck [3] to Cavaleiro and Alizadeh [1, 2]. The last of these reports $d = 5 \cdot 10^3$ at $n = 10^5$ in MATLAB, against $d = 1.6 \cdot 10^4$ at $n = 10^3$ here; the two sweep different axes and are not directly comparable. Gärtner and Schönherr's reservation that a QP formulation needed exact arithmetic past a few hundred dimensions Table 2 bears on without settling: in double precision the method returns a ball out to $d = 1.6 \cdot 10^4$ whose enclosure error is a few ulp, on Gaussian clouds, which are not the inputs that reservation was about. The QP machinery is standard [9]. An implementation of the method, of the pivoting method of Section 4, of the Goldfarb–Idnani core, and of the conic front end is available at <https://github.com/tschm/cvxball>, where the two methods this note compares ship side by side with the Clarabel and Welzl routes beside them as references.

References

- [1] M. Cavaleiro and F. Alizadeh. A faster dual algorithm for the Euclidean minimum covering ball problem. *Annals of Operations Research*, 2018. DOI 10.1007/s10479-018-3123-5.
- [2] M. Cavaleiro and F. Alizadeh. A dual simplex-type algorithm for the smallest enclosing ball of balls. *Computational Optimization and Applications* 79(3):767–787, 2021.
- [3] P. M. Dearing and C. R. Zeck. A dual algorithm for the minimum covering ball problem in $\mathbb{R}^n$. *Operations Research Letters* 37(3):171–175, 2009.
- [4] J. Elzinga and D. W. Hearn. Geometrical solutions for some minimax location problems. *Transportation Science* 6(4):379–394, 1972.
- [5] K. Fischer, B. Gärtner and M. Kutz. Fast smallest-enclosing-ball computation in high dimensions. *Proc. ESA*, 630–641, 2003.
- [6] B. Gärtner. Fast and robust smallest enclosing balls. *Proc. ESA*, 325–338, 1999.
- [7] B. Gärtner and S. Schönherr. An efficient, exact, and generic quadratic programming solver for geometric optimization. *Proc. 16th Annu. ACM Sympos. Comput. Geom.*, 110–118, 2000.
- [8] Hexaly. Smallest circle problem. Modelling template, <https://www.hexaly.com/templates/smallest-circle-problem>, accessed September 2026.
- [9] J. Nocedal and S. J. Wright. *Numerical Optimization*, 2nd ed. Springer, 2006.
- [10] T. Schmelzer and M. Stoll. Goldfarb–Idnani revisited: invariants, certificates, and the limits of guessing. arXiv:2608.30933, 2026.
- [11] T. Schmelzer and M. Stoll. Non-negative conjugate gradients. arXiv:2607.22121, 2026.
- [12] J. J. Sylvester. A question in the geometry of situation. *Quarterly Journal of Pure and Applied Mathematics* 1:79, 1857.
- [13] E. Welzl. Smallest enclosing disks (balls and ellipsoids). *New Results and New Trends in Computer Science*, LNCS 555, 359–370, 1991.
- [14] P. Wolfe. Finding the nearest point in a polytope. *Mathematical Programming* 11:128–149, 1976.